

Lesson: Hyperparameter Tuning Methods

Imagine building a high-performance sports car. You've got the engine, the tires, the sleek design, but if you don't tune it correctly, it won't win any races. Machine learning models work the same way. You can choose the best algorithm, feed it good data, and still get poor results. Unless you tune it right.

That's where hyperparameter tuning comes in. It's like fine-tuning your model's "settings" to unlock its full potential. Have you ever wondered:

- Why do two students using the same algorithm get very different results?
- How Kaggle winners boost their scores without changing their dataset?
- Why does a model perform great on one dataset but terribly on another?

The answer often lies in the hyperparameters, the knobs and dials we control before training begins.

Learning Outcomes

By the end of this lesson, you will be able to:

- Importance of hyperparameter tuning.
- The difference between GridSearchCV and RandomizedSearchCV.
- Tuning hyperparameters for Random Forest and SVM models

Introduction to Hyperparameter Tuning

Before diving into **hyperparameter tuning**, it is important to distinguish between two closely related concepts in machine learning: **parameters** and **hyperparameters**. Both influence model performance, but they serve different roles in the learning process.

What Is a Parameter?

A parameter is an internal configuration of a machine learning model that is learned or estimated directly from the training data. These values determine the model's performance and are adjusted during the training process to minimize error or maximize accuracy.

Key Characteristics of Parameters:

- **Part of the Model:** Parameters are embedded within the model architecture.
- **Learned During Training:** They are optimized based on the input data using learning algorithms.
- **Crucial for Inference:** Parameters determine the model's output for a given input.
- **Directly Affect Performance:** Well-learned parameters improve predictive accuracy; poor ones lead to high error.

Examples:

The weights and biases in a neural network, the coefficients in a linear regression model, the split thresholds in a decision tree.

What Are Hyperparameters?

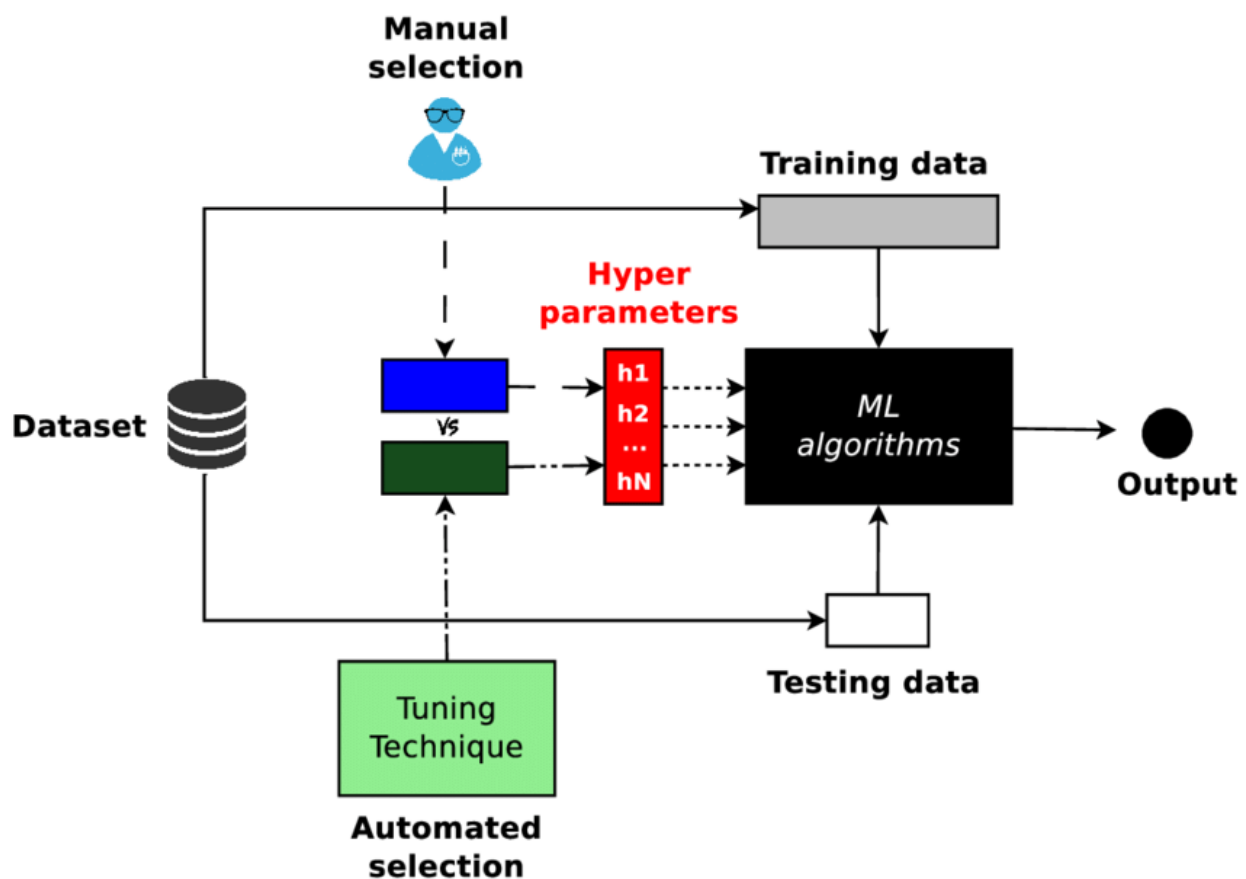
In contrast, A hyperparameter is a configuration that is set before training begins and cannot be learned from the data directly. Hyperparameters control the training process and model structure, and their values can significantly impact performance.

Key characteristics of hyperparameters include:

- **Externally Set:** They are configured by the practitioner rather than estimated from data.
- **Guide the Learning Process:** Hyperparameters control model behavior—such as how fast it learns, how complex it can become, or how regularized it is.
- **Influence Model Capacity:** They determine whether a model can capture complex patterns or remains too simple.
- **Require Tuning:** Optimal hyperparameters vary by dataset and must often be tuned through experimentation.
- **Can Be Tuned Automatically:** Techniques like **grid search**, **randomized search**, and **Bayesian optimization** are used to automate this process.

Examples:

The number of trees in a Random Forest (`n_estimators`), the learning rate in Gradient Boosting or Neural Networks (`learning_rate`), the regularization parameter in SVM (`C`), the maximum depth of a decision tree (`max_depth`).



Identifying Hyperparameters

Choosing the right hyperparameters is key to improving a model's performance and accuracy. However, there are no fixed rules or default values that guarantee the best results. To find the best combination, you need to experiment, which is called hyperparameter tuning or hyperparameter optimization. In short, hyperparameter tuning is the process of finding the best values for a model's hyperparameters to make it perform well.

Importance of Hyperparameter Tuning

Hyperparameters play a critical role in determining a machine learning model's **effectiveness, efficiency, and generalization**. Unlike model parameters, which are learned from data, hyperparameters must be carefully selected and tuned to ensure optimal performance. Poorly chosen hyperparameters can lead to two common issues:

- **Underfitting:** The model is too simple to capture underlying patterns.
- **Overfitting:** The model learns noise and spurious correlations in the training data.

Impact of Hyperparameter Tuning

Hyperparameter tuning allows practitioners to **optimize model performance** by systematically exploring the configuration space. Adjusting hyperparameters can improve accuracy, speed up training, reduce overfitting, and help the model generalize better to unseen data. Think of tuning as **calibrating a machine**—not changing the mechanism itself, but fine-tuning the controls to get the best output.

Illustrative Example: Learning Rate

Consider the **learning rate**, a common hyperparameter in gradient-based models (e.g., neural networks):

- **Too high:** The model updates too aggressively and may overshoot optimal solutions, leading to instability or failure to converge.
- **Too low:** The model learns too slowly, possibly never reaching a good solution within a reasonable number of training steps.
- **Well-tuned:** Training is stable, efficient, and converges to a high-performing solution.

This balance must be found **experimentally**, as the ideal value depends on the dataset, model architecture, and other hyperparameters.

Summary

- Hyperparameter tuning is a **core step in the machine learning pipeline**.
 - It has a **direct impact** on model behavior and performance.
 - Effective tuning can mean the difference between a mediocre model and one that performs at or near the state of the art.
-

Balancing Bias and Variance through Hyperparameter Tuning

A key objective of hyperparameter tuning is to strike the right balance between bias and variance. Bias refers to errors when a model's predictions deviate significantly from actual outcomes. This often happens when a model is underfitted, meaning it's too simple to capture the underlying patterns in the data.

On the other hand, variance describes how sensitive a model is to fluctuations in the training data. A model with high variance is typically overfitted; it performs well on the training data but fails to generalize to unseen data, leading to poor performance on test datasets.

Effective hyperparameter tuning helps minimize both bias and variance, leading to a model that is accurate and consistent. Additionally, it ensures that computational resources are used efficiently during the training process.

Think of bias and variance like drawing a picture of a tree. **High bias** (underfitting) is like drawing a single straight line for the tree—it's too simple and misses the branches and leaves (the patterns in the data). **High variance** (overfitting) is like drawing every tiny detail of the tree, including the exact shape of every leaf—it's so specific that it doesn't look like other trees (new data). Hyperparameter tuning helps you draw a picture that's detailed enough to capture the tree's shape but simple enough to apply to other trees, balancing accuracy and flexibility.

Process of Hyperparameter Tuning

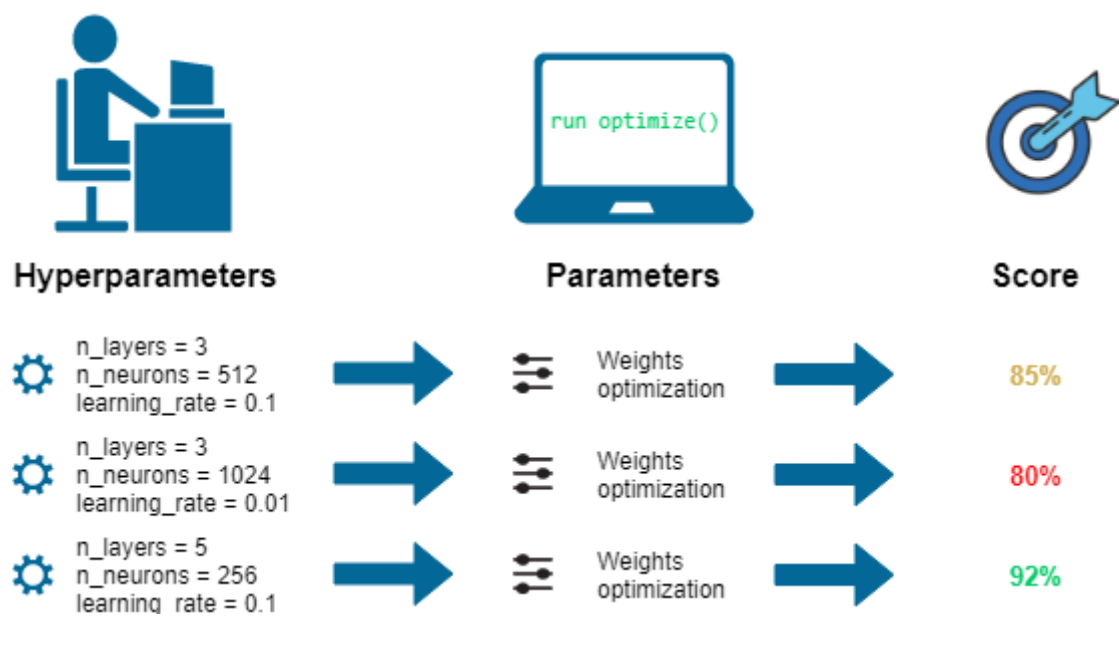
As mentioned earlier, hyperparameter tuning can be done manually or automatically.

- **Manual tuning:** Involves changing values by hand and testing results. It takes time, but it helps you understand how each setting affects your model.
- **Automated tuning:** Uses algorithms to test many combinations for you.

The process is repetitive: you try different settings and check how the model performs. Usually, you decide on a performance goal, like accuracy, and aim to increase or decrease it. To avoid testing on just one part of the dataset, it's a good practice to use cross-validation, which tests the model on different data splits to make sure it works well overall.

Cross-validation

It's like testing a chef's recipe by having different groups of people taste it, instead of just one group. It splits your data into multiple parts (e.g., five groups), trains the model on some parts, and tests it on others, repeating this for each group. This ensures the model performs well across different data samples, not just one, helping you trust that it will work on new, unseen data.



Methods for Hyperparameter Tuning

Once key hyperparameters have been identified, the next step is to explore different values to find the combination that yields the best model performance. This process—known as **hyperparameter optimization**—can be approached in several ways, ranging from manual experimentation to sophisticated automated techniques.

1. Manual Search

Manual search is the simplest method of tuning hyperparameters. You choose values for each hyperparameter manually, train the model, and evaluate its performance. You repeat this process with different values until you find a good combination.

Pros	Cons
Simple and easy to implement	Very time-consuming

Pros	Cons
Useful when you know the model well	Not scalable for many parameters
Leverages domain intuition	May miss optimal combinations due to limited coverage

For example, suppose you're working on a medical dataset and know that simpler models often perform better due to noisy data. In that case, you might manually set a lower `max_depth` in a Random Forest to keep the model less complex. This hands-on approach lets you use your knowledge to guide the tuning process.

Syntax of Manual

```
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# Manually trying different C and kernel values
model = SVC(C=1.0, kernel='rbf') # you can try 'linear', 'poly', etc.
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
```

2. Grid Search

GridSearch is commonly seen as a brute force method for tuning hyperparameters. It tries out every possible combination of specified values for each hyperparameter. A grid is formed with all these value combinations, and the model is trained using each one. The performance for every combination is recorded, and the one that gives the best outcome is chosen.

Imagine you're searching for the best recipe by trying every possible combination of ingredients in exact amounts, like testing 1 tsp, 2 tsp, and 3 tsp of sugar with 1 cup or 2 cups of flour. Grid Search does this for hyperparameters, checking every combination you specify to find the best one.

Pros	Cons
Exhaustively evaluates all parameter combinations	Computationally expensive with large search space
Guarantees best result in the defined grid	Inefficient if many parameters are irrelevant
Works well with small discrete parameter sets	Takes longer as number of parameters increases

Example: Testing all combinations of $C = [0.1, 1, 10]$ and $\gamma = [0.01, 0.1, 1]$ in an SVM model leads to 9 training runs.

Syntax Example: Grid Search with `GridSearchCV`

The following code demonstrates how to apply **grid search** using scikit-learn's `GridSearchCV` on a `RandomForestClassifier`. It performs a systematic evaluation of all hyperparameter combinations using cross-validation.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

# Define parameter grid
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

# Initialize model
rf = RandomForestClassifier(random_state=42)

# Grid search with 5-fold cross-validation
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, cv=5)
grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)
```

Result: Best Parameters: {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 100}. Best Score: 0.9458498023715414

- This example demonstrates:
 - How to define a search grid with multiple hyperparameters.
 - The integration of cross-validation (`cv=5`) to ensure robust evaluation.
 - How to extract the best-performing configuration and corresponding score.

3. Randomized Search

Picture searching for treasure in a huge field. Instead of checking every single spot (like Grid Search), Randomized Search picks a few random spots to dig, hoping to find the treasure faster. It's less thorough but can explore a bigger area with less effort.

The Randomized Search method chooses hyperparameter values randomly, unlike `GridSearchCV`, which tests all fixed combinations. In every iteration, `RandomizedSearchCV` tries a new random set of hyperparameters and tracks how well the model performs. After running for a set number of iterations, it selects the combination that gave the best result. This method is often more efficient, as it can cover a broader range of values with fewer computations.

Pros and cons compared to Grid Search:

Pros	Cons
Faster than grid search for large parameter spaces	Might miss the optimal combination
Can sample from distributions instead of fixed values	Results may vary due to randomness
Can find good results with fewer iterations	Harder to interpret coverage of parameter space

Syntax Example: Random Search with **RandomizedSearchCV**

Below is an example using **RandomizedSearchCV** to tune an SVM classifier. This method randomly samples 10 hyperparameter combinations from the specified distributions.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import uniform

# Define distributions instead of a full grid
param_dist = {
    'C': uniform(0.1, 10),
    'gamma': ['scale', 'auto'],
    'kernel': ['linear', 'rbf']
}

svm = SVC()

# Randomized search with 10 iterations
random_search = RandomizedSearchCV(estimator=svm,
                                   param_distributions=param_dist,
                                   n_iter=10, cv=5, random_state=42)
random_search.fit(X_train, y_train)

print("Best Parameters:", random_search.best_params_)
print("Best Score:", random_search.best_score_)
```

Output

```
Best Parameters: {'C': 1.9182496720710063, 'gamma': 'scale', 'kernel':
'linear'}
Best Score: 0.9727272727272727
```

Comparing **GridSearchCV** vs. **RandomizedSearchCV**

Both `GridSearchCV` and `RandomizedSearchCV` are powerful tools in scikit-learn for hyperparameter optimization using cross-validation. While they share the same goal—finding the best combination of hyperparameters—they differ significantly in strategy, efficiency, and scalability. Understanding their differences helps practitioners choose the right tool based on model complexity, available compute resources, and time constraints.

Conceptual Differences

- `GridSearchCV` exhaustively searches across **all** possible combinations in a predefined grid of hyperparameter values.
- `RandomizedSearchCV` samples a **fixed number of combinations** from specified distributions or lists.

Comparison Table

Feature	<code>GridSearchCV</code>	<code>RandomizedSearchCV</code>
Search Strategy	Exhaustive: tries every combination	Random sampling of a fixed number of combinations
Best Use Case	Small, discrete hyperparameter spaces	Large or continuous hyperparameter spaces
Flexibility	Limited to fixed value grids	Allows probability distributions for continuous ranges
Computational Efficiency	Often expensive as search space grows	More efficient with large or redundant spaces
Reproducibility	Fully deterministic	Depends on random seed (can be made reproducible)
Guarantees Best in Grid?	Yes, within the defined grid	No, might miss optimal combination
Scalability	Poor with many parameters	Scales better to high-dimensional spaces
Custom Search Depth	Not supported (all combinations tried)	Tunable via <code>n_iter</code> to control cost/performance tradeoff

Summary

Use `GridSearchCV` when:

- You have a **small**, well-defined set of hyperparameters.
- You want **exhaustive evaluation** and have sufficient resources.

Use `RandomizedSearchCV` when:

- The hyperparameter space is **large or continuous**.
- You need a **faster, approximate solution**.

- You're working under **limited time or compute budgets**.
-

Model-Specific Tuning Examples

Hyperparameter tuning is not one-size-fits-all. Each model type has a distinct set of hyperparameters that control its behavior, complexity, and training process. Understanding what to tune and how it affects performance is essential for building effective models. We begin with one of the most widely used ensemble methods: **Random Forest**.

Hyperparameter Tuning for Random Forest

Random Forest is a robust ensemble learning method that builds multiple decision trees and aggregates their predictions for classification or regression. Tuning its hyperparameters helps optimize performance, prevent overfitting, and reduce unnecessary computation.

Key Hyperparameters for Random Forest

Tuning the following hyperparameters can significantly improve a Random Forest model's **accuracy**, **training efficiency**, and **generalization performance**. The key hyperparameters are:

- `n_estimators`
- `max_depth`
- `max_features`
- `min_samples_split`

1. `n_estimators`

The `n_estimators` parameter specifies the number of decision trees to be built in a Random Forest ensemble.

- A **larger number of trees** typically leads to **better model performance** by reducing variance and stabilizing predictions across multiple random samples.
- However, increasing this value also results in **greater computational cost**—both in training time and memory usage.
- A **smaller number of trees** may speed up the process, but risks **underfitting** due to insufficient model complexity.

Recommendation:

Start with a moderate number (e.g., 100–300) and increase gradually if the model shows signs of instability or high variance. Use cross-validation to evaluate the trade-off between performance gain and computational overhead.

2. `max_depth`

The `max_depth` parameter defines the **maximum depth** of each decision tree in the Random Forest.

- A **high value** allows trees to grow deeper, enabling the model to **capture complex relationships** and finer patterns in the data. However, this can lead to **overfitting**, especially when the training data is noisy or limited.
- A **low value** restricts the depth, producing **simpler trees** that are less likely to overfit, but they may also **underfit** if the model fails to capture important structure.

Recommendation:

Set a reasonable maximum depth (e.g., 10–30) to strike a balance between model expressiveness and generalization. Use cross-validation to find the depth that minimizes validation error without overfitting.

3. `max_features`

The `max_features` parameter controls **how many features** the model considers when looking for the **best split** at each decision node within a tree.

This hyperparameter directly affects:

- **Tree diversity:** By reducing the number of features considered at each split, you encourage different trees to make different decisions.
- **Training speed:** Fewer features per split mean faster training.
- **Model complexity:** Using all features can increase the risk of overfitting.

Common Options:

- `'sqrt'`: Uses the square root of the total number of features. This is the **default for classification** tasks and offers a good balance of diversity and performance.
- `'log2'`: Uses the base-2 logarithm of the number of features. This **increases randomness** and is more conservative.
- `None`: Considers **all features** at every split, which may improve individual tree performance but increases the risk of **overfitting** and slows training.

Recommendation:

For classification problems, use `'sqrt'` as a reliable default. It tends to reduce correlation between trees and improve generalization while keeping computation efficient.

4. `min_samples_split`

The `min_samples_split` parameter sets the minimum number of samples required to split an internal node in a decision tree.

This hyperparameter helps regulate the growth and complexity of each tree in the forest.

- A **higher value** encourages the model to create **simpler and more generalized trees**, reducing the risk of overfitting by preventing splits on small, potentially noisy subsets.
- A **lower value** allows the tree to **split more frequently**, enabling it to capture **finer details** in the data, which can lead to **overfitting**, especially with small datasets.

Recommendation:

Start with the default value of 2, and increase it (e.g., to 5 or 10) when your model shows signs of overfitting or when working with noisy data. Use cross-validation to tune this value for the best generalization.

Summary of Key Random Forest Hyperparameters

Hyperparameter	What It Controls	Effect of Low Value	Effect of High Value	Tuning Tip
n_estimators	Number of decision trees in the forest	Faster, less stable model (underfitting)	More stable, less variance, slower training	Start with 100–300 and increase if variance is high
max_depth	Maximum depth of each tree	Underfitting due to shallow trees	Overfitting by memorizing noise	Tune to balance complexity and generalization (e.g., 10–30)
max_features	Number of features considered when splitting	Increases tree diversity, may reduce accuracy	Reduces diversity, may overfit	Use 'sqrt' for classification as a balanced default
min_samples_split	Minimum samples required to split a node	More splits, high complexity (overfitting risk)	Fewer splits, simpler trees (better generalization)	Raise to 5 or 10 if trees are growing too deep

This table reinforces the key trade-offs involved in tuning each hyperparameter and provides practical guidance for setting initial values and adjustments.

Example: Hyperparameter Tuning in Random Forest Using RandomizedSearchCV

The following example demonstrates how to perform hyperparameter tuning for a Random Forest Classifier using Randomized Search on the Iris dataset. This method is especially effective when the hyperparameter space is large, and exhaustive search would be computationally expensive.

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from scipy.stats import randint

# Load dataset
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Define the model
rf = RandomForestClassifier(random_state=42)

# Hyperparameter distribution
param_dist = {
    'n_estimators': randint(50, 200),
    'max_depth': randint(3, 20),
    'max_features': ['sqrt', 'log2', None], #
    'min_samples_split': randint(2, 10),
    'min_samples_leaf': randint(1, 5)
}

# RandomizedSearchCV
random_search = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist,
    n_iter=20,
    cv=5,
    scoring='accuracy',
    random_state=42,
    n_jobs=-1
)

# Fit and print results
random_search.fit(X_train, y_train)
print("Best parameters found: ", random_search.best_params_)
print("Best cross-validation accuracy: ", random_search.best_score_)

best_rf = random_search.best_estimator_
test_accuracy = best_rf.score(X_test, y_test)
print("Test set accuracy: ", test_accuracy)

```

```

Best parameters found:
{'max_depth': 12, 'max_features': 'log2', 'min_samples_leaf': 1,
 'min_samples_split': 8, 'n_estimators': 195}

```

```

Best cross-validation accuracy: 0.9428571428571428
Test set accuracy: 1.0

```

Output

```
Best parameters found:
{'max_depth': 12, 'max_features': 'log2', 'min_samples_leaf': 1,
 'min_samples_split': 8, 'n_estimators': 195}

Best cross-validation accuracy: 0.9428571428571428
Test set accuracy: 1.0
```

Key Takeaways

- **RandomizedSearchCV** efficiently explores the hyperparameter space using random sampling.
- It can handle continuous distributions (via **scipy.stats**) and categorical options.
- Cross-validation ensures the model's generalization is assessed during tuning.
- The final test accuracy confirms the effectiveness of the optimized model on unseen data.

Hyperparameter Tuning for SVM

Support Vector Machines (SVMs) are powerful and flexible machine learning models used in classification, regression, and outlier detection. Like many models, SVMs depend heavily on several key hyperparameters that directly influence their performance. Understanding and tuning these hyperparameters is critical to achieving a good balance between bias and variance.

Key Hyperparameters for SVM

- **C** – Regularization parameter
- **kernel** – Choice of transformation function
- **gamma** – Kernel coefficient (for 'rbf', 'poly', and 'sigmoid' kernels)

1. C (Regularization Parameter)

The **C** parameter determines the trade-off between a smooth decision boundary and correctly classifying training examples.

- **Small value of C**

Allows the model to prioritize a smoother margin, even if it means misclassifying more training points. This results in a simpler model with potentially better generalization (but higher bias).

- **Large value of C**

Tries to classify all training examples correctly, even if that leads to a complex decision boundary. This might improve training accuracy but risks **overfitting** (high variance).

Think of C as how strictly the model enforces rules. A small C is like a lenient teacher who allows some errors for the sake of simplicity. A large C is a strict teacher who wants perfection, even if it

leads to complications.

2. Kernel

The `kernel` function defines how input data is transformed or mapped into a higher-dimensional space so that it can be separated more easily. Choosing the right kernel is critical to aligning the model’s capacity with the complexity of the underlying data.

Common Kernel Types:

Kernel	Description	When to Use
<code>linear</code>	No transformation; finds a straight-line separator.	Best for linearly separable data.
<code>poly</code>	Maps features into a polynomial space (degree can be tuned).	When data shows polynomial relationships.
<code>rbf</code>	Radial Basis Function (Gaussian kernel); handles local patterns and curvature.	A strong default for non-linear problems.
<code>sigmoid</code>	Similar to neural network activation functions.	Rarely used; niche applications.

Recommendation:

Start with the `rbf` kernel for non-linear classification problems. Use `linear` if the dataset is simple, high-dimensional, or if interpretability is a concern.

3. Gamma (γ)

The `gamma` parameter defines the influence radius of a single training example when using non-linear kernels like RBF, polynomial, or sigmoid. It determines how much each data point affects the shape of the decision boundary.

- High value of `gamma`**
Causes the influence of each data point to be very localized, resulting in a tight, complex boundary that closely fits the training data. This increases the risk of overfitting.
- Low value of `gamma`**
Allows the influence of each data point to extend farther, producing a smoother, more general boundary. This can result in underfitting if the model fails to capture important local patterns.

Summary of Key SVM Hyperparameters

Hyperparameter	What It Controls	Effect of Low Value	Effect of High Value	Tuning Tips
----------------	------------------	---------------------	----------------------	-------------

Hyperparameter	What It Controls	Effect of Low Value	Effect of High Value	Tuning Tips
C	Regularization strength (controls margin vs. misclassification)	Allows more misclassifications (simpler model, higher bias)	Forces perfect classification (complex model, risk of overfitting)	Start with C = 1 . Increase to reduce bias; decrease to reduce variance.
kernel	Method for transforming data into higher-dimensional space	N/A (choose based on data type and complexity)	N/A	Use ' linear ' for simple/high-dimensional data; ' rbf ' for general non-linearity
gamma	Influence of single data points in non-linear kernels	Broad influence; smoother, simpler decision boundary (risk of underfitting)	Narrow influence; complex, sensitive boundaries (risk of overfitting)	Start with ' scale ' (default); adjust based on model complexity and overfitting

Example: Hyperparameter Tuning in SVM Using **GridSearchCV**

This example demonstrates how to tune hyperparameters for a **Support Vector Machine (SVM)** using **GridSearchCV** on the **Iris dataset**. Grid search systematically evaluates all combinations of defined hyperparameter values using cross-validation.


```

from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load example dataset
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# Define the SVM model
svm = SVC()

# Define the parameter grid
param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf', 'poly'],
    'gamma': ['scale', 'auto']
}

# Setup GridSearchCV
grid_search = GridSearchCV(svm, param_grid, cv=5)
grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)

```

Output

```

Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'linear'}
Best Score: 0.9640316205533598

```

Key Takeaways

- Grid search evaluates every combination in the parameter grid using **5-fold cross-validation**.
- The best-performing configuration is chosen based on cross-validated accuracy.
- In this example, the **linear kernel with C = 1** and **gamma = 'scale'** achieved the highest validation performance.

Conclusion

Hyperparameter tuning is the art and science of optimizing machine learning models by adjusting their key settings before training begins. Just like tuning a high-performance engine, tweaking the right hyperparameters can significantly boost model accuracy and generalization. In this lesson, we explored the difference between parameters and hyperparameters, emphasized the importance of

tuning to balance bias and variance, and introduced three common tuning methods: Manual Search, Grid Search, and Randomized Search.

We also applied these methods to real models like Random Forest and SVM, showing how even small changes in hyperparameters (like `C`, `max_depth`, or `n_estimators`) can lead to big performance improvements.